

```
≡ output.txt X
≡ output.txt
1 Area of Rectangle 5
2 Perimeter of Rectangle 12
3 Area of Circle 314.25
4 Perimeter of Circle 62.85
```

Friend Function and Friend Classes in C++

A friend function in C++ is defined as a function that can access private, protected, and public members of a class. A friend function can be a member of another class or can be a global function in C++.

Note :- A friend function of a class is defined outside that class scope but it has the right to access all private, protected, and public members of the class. Even though the prototypes for friend functions appear in the class definition, friends are not member functions.

Characteristics of a Friend Function in C++:

- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- It cannot access the member names directly and has to use an object name and dot membership operator with the member's name.
- It can be declared either in the private or the public part.
- The function is not in the scope of the class of which it has been declared as a friend



```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class Test{
4  private:
5      int x;
6  protected:
7      int y;
8  public:
9      int z;
10     friend void Fun();
11 };
12 void Fun(){
13     Test t;
14     t.x = 10, t.y = 20, t.z = 30;
15     cout << " X : " << t.x << endl;
16     cout << " Y : " << t.y << endl;
17     cout << " Z : " << t.z << endl;
18 }
19 int main(){
20     Fun();
21     return 0;
22 }
23 //Output
24 X : 10
25 Y : 20
26 Z : 30
```

Friend Classes in C++:

In C++, a friend class can access private, protected, and public members of another class in which it is declared a friend. It is sometimes useful to allow a particular class to access private members of other classes.

Instead of writing the name before the My Class definition, we can also write `friend class Your;`

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class My{
4  private:
5      int x;
6  protected:
7      int y;
8  public:
9      int z;
10     friend class Your;
11 };
12 class Your{
13 public:
14     My m;
15     void Fun(){
16         m.x = 10;
17         m.y = 20;
18         m.z = 30;
19         cout << "X = " << m.x << endl;
20         cout << "Y = " << m.y << endl;
21         cout << "Z = " << m.z << endl;
22     }
23 };
24 int main(){
25     Your obj;
26     obj.Fun ();
27     return 0;
28 }
```

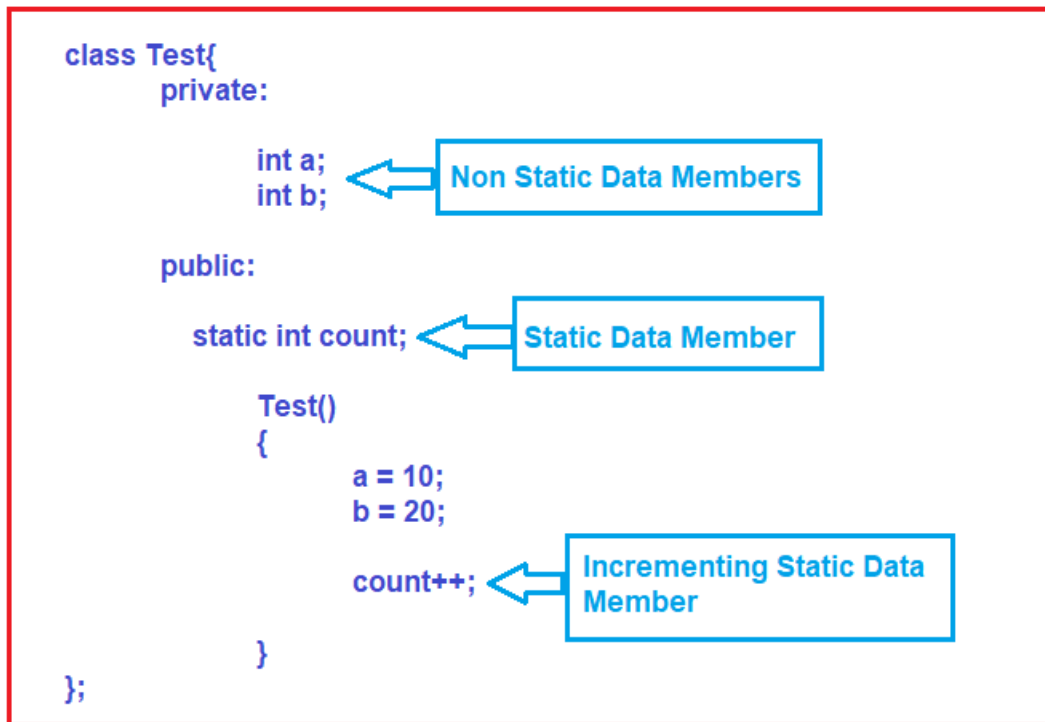
Key Points of Friend Functions and Classes in C++ :-

- Friend functions are global functions
- They can access private, protected, and public members of a class upon their objects
- A class can be declared as a friend of another class
- All the functions of the friend class can access private and protected members of other classes.
- **Friendship is not mutual.** If class A is a friend of B, then B doesn't become a friend of A automatically.
- **Friendship is not inherited.**
- Friends should be used only for a limited purpose.
- Too many functions or external classes are declared as friends of a class with protected or private data, which is against the principle of encapsulation in object-oriented programming.

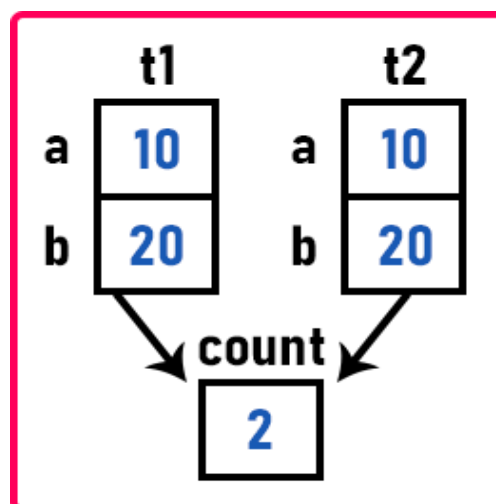
Static Data Members in C++ :-

We can define class members' static using the static keyword in C++. When we declare a member of a class as static **it means no matter how many objects of the class are created, there is only one copy of the static member available throughout the program. So static variables or static data members of a class belong to a class. That doesn't belong to an object.**

It means when we make a variable as static, that variable will occupy memory only once. And all the objects can share it.



One more thing we have to address syntactically. When we have a static variable inside the class, **you must have declared it outside again**. So, when we declare the class variable outside the class, then it will become a global variable but we want it to be accessible only inside the `Test` class. So, we can use the scope resolution operator to make it accessible only inside the `Test` class.





```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class Test{
4  private:
5      int a;
6      int b;
7  public:
8      static int count;
9      Test(){
10         a = 10, b = 20;
11         count++;
12     }
13 };
14 int Test::count = 0;
15 int main(){
16     Test t1, t2;
17     cout<<t1.count<<endl;
18     cout<<t2.count<<endl;
19     cout<<Test::count<<endl;
20     return 0;
21 }
22 //Output
23 2
24 2
25 2
```

Static Member Functions in C++ :- By declaring a member function as static in C++, we make it independent of any particular object of the class. A static member function can be called even if no objects of the class exist and the static functions are accessed using the class name and the scope resolution operator (::) and even we can also access them using objects of the class.

Note :- the important thing is Static Member Functions in C++ can access only static data members of a class. They cannot access non-static data members. So static member functions also belong to a class.

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class Test{
4  public:
5      int a;
6      static int count;
7      Test(){
8          a = 10;
9          count++;
10     }
11     static int getCount(){
12         return count;
13     }
14 };
15 int Test::count = 0;
16 int main(){
17     cout<<"Calling count without object : "<<Test::count<<endl;
18     cout<<"Calling getCount without object : "<<Test::getCount()<<endl;
19     Test t1;
20     cout<<"Calling count with object t1 : "<<t1.count<<endl;
21     cout<<"Calling getCount with object t1 : "<<t1.getCount()<<endl;
22 }
23 //Output
24 Calling count without object : 0
25 Calling getCount without object : 0
26 Calling count with object t1 : 1
27 Calling getCount with object t1 : 1
```

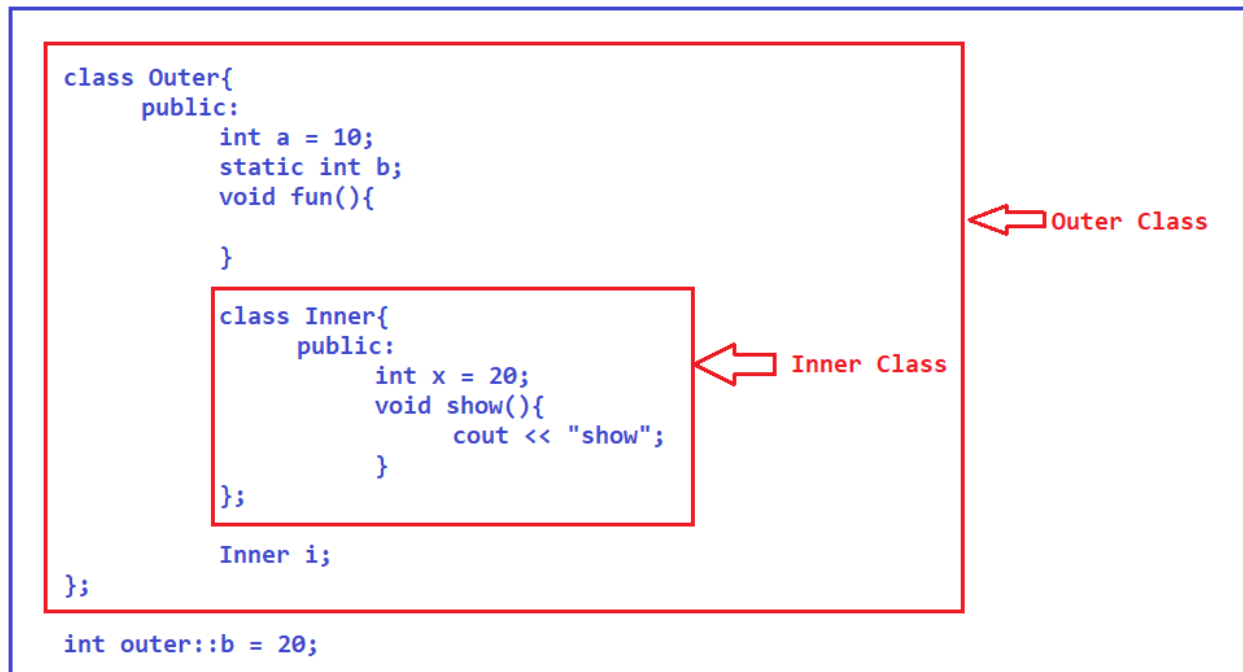
Key Points on Static Members in C++:

- Static data members are members of a class.
- Only one instance of static members is created for the entire class and shared by all objects of that class, no matter how many objects are created.
- It is initialized before any object of this class is created, even before the main starts.
- They can be accessed directly using the class name.
- Static member functions are functions of a class, they can be called using the class name, without creating the object of a class.
- They can access only static data members of a class; they cannot access non-static members of a class.
- It is visible only within the class, but its lifetime is the entire program.

Inner Class in C++ :-

Writing a class inside another class is called an **inner class**. It is useful only within that class. **If you are writing a class that is not useful everywhere or limited to one class.** Then you can write a class inside of another one.

So, it is just like reducing the complexity of a bigger class, we can write smaller classes inside.



Note :- Can we create this object before the declaration of the Inner class? No, it must be done after the definition.

- Inner class can access the members of the Outer class if they are static.
- Can the Outer class create the object of the Inner class? Yes, it can. Now using that object can it access all the members of a class
- We can access only those members which are public. We cannot access private and protected members of the Inner class.

When to use Inner Classes in C++?

```
class LinkedList{
    class Node{
        int data;
        Node *next;
    };
    Node *Head;
    ----
    ----
};
```



```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class Outer{
4  public:
5      void Fun (){
6          i.Display();
7      }
8      class Inner{
9      public:
10         void Display(){
11             cout<<"Display of Inner"<<endl;
12         }
13     };
14     Inner i;
15 };
16 int main(){
17     Outer::Inner i;
18     i.Display();
19     return 0;
20 }
21 //Output
22 Display of Inner
```